PROJECT REPORT

MERN-BASED BLOOD DONOR FINDER

A Web Application for Finding and Managing Blood Donors

Submitted By
G PANNAGESH
Roll No: 25MVCSMR0486

Department of Computer Science
Academic Project Report

2026

Abstract

The MERN-Based Blood Donor Finder is a web application designed to help users locate potential blood donors according to blood group and location. The system uses MongoDB to store donor information, Express.js and Node.js to provide backend REST APIs, and React.js to deliver an interactive user interface. The application is planned to support donor registration, viewing donor records, searching and filtering, updating details, and removing or deactivating records when appropriate. Authentication and role-based access can be used to protect donor data and administrative functions. The purpose is to make donor information easier to organize and search while emphasizing consent, privacy, and responsible use of contact details.

1. Introduction

Blood is required in emergencies, surgeries, childbirth complications, and the treatment of several medical conditions. Finding a compatible donor quickly can be challenging when donor information is scattered across personal contacts, messages, or informal groups. A centralized donor directory can make searching more organized.

This project proposes a web-based blood donor finder built with the MERN stack: MongoDB, Express.js, React.js, and Node.js. Users can search donor records by blood group and location, while authorized users can maintain accurate records. The application is intended to support discovery of potential donors; it does not replace medical screening, blood-bank procedures, or professional confirmation of blood compatibility.

2. Problem Statement

Develop a MERN-based blood donor finder that allows users to find potential donors by blood group and location. The system should manage donor name, blood group, location, and contact information in MongoDB. React.js should provide the frontend, while Node.js and Express.js should expose backend APIs for creating, viewing, updating, searching, and managing donor records.

3. Objectives

- Demonstrate full-stack web development using the MERN stack.
- Store and manage donor records using MongoDB.
- Implement CRUD operations through RESTful APIs.
- Build a responsive and user-friendly interface with React.js.
- Provide search and filtering by blood group and location.

- Add authentication and role-based authorization where required.
- Present project information and results in a clear, accessible format.
- Protect personal data and encourage donor consent and accurate information.

4. Scope of the Project

The initial scope includes donor registration, donor listing, filtering by blood group, location-based search, editing donor details, and controlled removal or deactivation of records. An administrator may manage inappropriate or outdated entries. Optional extensions include verified donor status, availability updates, request notifications, and an emergency contact workflow. The application should not publicly expose a donor's phone number without appropriate consent.

5. Technologies Used

Technology	Role	Purpose
MongoDB	Database	Stores donor profiles and user accounts.
Express.js	Backend framework	Defines API routes and middleware.
React.js	Frontend library	Builds pages, forms, search, and donor listings.
Node.js	Runtime	Runs the JavaScript backend server.
Mongoose	ODM	Defines schemas and validates MongoDB records.
JWT / secure sessions	Authentication	Supports authenticated access where configured.
CSS / responsive UI	Presentation	Provides layout and usability across screen sizes.

6. System Architecture

The application follows a three-layer web architecture. The React frontend sends HTTP requests to the Express API. Express routes validate requests and invoke controller logic. The backend uses Mongoose to read or update documents in MongoDB, then returns a JSON response to React.

- Presentation layer: React components for registration, login, donor search, donor details, and administration.
- Application layer: Node.js and Express.js routes, controllers, validation, authentication, and error handling.
- Data layer: MongoDB collections for donor profiles and user accounts.

7. Main System Modules

7.1 Donor Registration

Collects donor name, blood group, location, contact details, and consent. Input validation should reject missing or invalid values.

7.2 Donor Search

Allows a user to filter donors by blood group and optionally narrow results by city, area, or postal code.

7.3 Donor Directory

Displays relevant donor records in a readable list or card layout, with only the details the viewer is authorized to see.

7.4 Update Donor Details

Allows the donor or an authorized administrator to update a record, such as location or contact information.

7.5 Record Management

Allows authorized users to deactivate or delete outdated, duplicate, or withdrawn records.

7.6 Authentication and Roles

Provides sign-in for protected actions. Roles may include donor/user and administrator; authorization must be checked by the backend.

7.7 Validation and Error Handling

Checks input formats, handles missing records, and returns clear API error messages.

8. Database Design

A possible Donor document structure is shown below. The exact fields can be adjusted to the final implementation.

```
{
"name": "Example Donor",
"bloodGroup": "O+",
"location": {
"city": "Hyderabad",
"area": "Example Area",
"postalCode": "000000"
},
"contact": "stored securely",
"availableToContact": true,
"consentToListing": true,
"isActive": true,
"createdAt": "timestamp",
"updatedAt": "timestamp"
}
```

Suggested validation: bloodGroup should be one of A+, A-, B+, B-, AB+, AB-, O+, or O-. Name and location should not be empty. Contact data should be validated and access-controlled. Timestamps help track record creation and updates. Do not store passwords as plain text; store a secure password hash if local account authentication is implemented.

9. REST API Design

Method	Endpoint	Purpose	Access
POST	/api/donors	Create a donor record	Authenticated or approved registration flow
GET	/api/donors	List/search donors; accepts bloodGroup and location filters	Public limited view or authenticated
GET	/api/donors/:id	View one donor record	According to privacy policy
PUT	/api/donors/:id	Update a donor record	Owner or administrator
PATCH	/api/donors/:id/status	Deactivate/reactivate a record	Owner or administrator
DELETE	/api/donors/:id	Delete a donor record when permitted	Owner or administrator
POST	/api/auth/register	Create an application account	Public, with validation

Method	Endpoint	Purpose	Access
POST	/api/auth/login	Authenticate an account	Public, rate-limited

The API list is a proposed design. Actual endpoints and access rules should match the code. The backend should validate all inputs, use suitable HTTP status codes, and avoid returning private fields unnecessarily.

10. User Interface Design

- Home page: introduces the purpose of the application and provides a clear donor-search entry point.
- Search page: provides a blood-group selector and location field, with readable results and empty-result messages.
- Donor registration page: presents a validated form and explains how contact information will be used.
- Donor management page: lets authorized users edit or deactivate their own records.
- Admin page: supports moderation of records and user access, if an administrator role is implemented.

11. Implementation Methodology

- Requirement analysis: define user roles, required fields, search behavior, and privacy rules.
- Database setup: create the MongoDB database and Mongoose schemas with validation.
- Backend development: configure Express, implement API routes/controllers, and add error-handling middleware.
- Frontend development: create React pages and reusable components for forms and search results.
- Integration: connect React to the backend using HTTP requests and handle loading, success, and error states.
- Testing: verify CRUD operations, filters, authorization, and responsive behavior.
- Deployment: configure environment variables, database access restrictions, HTTPS, and production logging.

12. Testing Plan

Test case	Expected result	Status
Register with valid donor details	Record is saved and confirmation is shown	To be tested
Submit a missing required field	Validation message appears; invalid record is not saved	To be tested
Search by blood group	Only matching eligible records are returned	To be tested
Search by blood group and location	Results match both selected filters	To be tested
Update a donor record	Authorized changes are saved and displayed	To be tested
Attempt unauthorized update	API rejects the request	To be tested
Deactivate a donor	Inactive record is excluded from normal search	To be tested
Search with no matches	A clear no-results message is displayed	To be tested

Testing status is marked 'To be tested' because this report describes the proposed system and does not claim that the application has already been implemented or tested.

13. Security, Privacy, and Responsible Use

- Obtain explicit consent before publishing a donor profile or making contact details available.
- Collect only the personal information needed for the service and provide a way to correct or remove it.
- Use HTTPS in deployment and keep database credentials and JWT/session secrets in environment variables.
- Hash passwords with a suitable password-hashing library; never store plaintext passwords.
- Enforce authorization on the server for every protected update, delete, and administrative action.
- Validate and sanitize user input, rate-limit authentication endpoints, and avoid exposing stack traces.
- Display a reminder that blood group information and donor availability must be confirmed through appropriate medical or blood-bank processes.
- Do not imply that a search result guarantees compatibility, availability, or medical suitability.

14. Expected Outcomes

When implemented, the system is expected to provide a centralized and searchable donor directory, demonstrate CRUD operations with MongoDB, expose REST APIs through Express.js and Node.js, and offer a React-based user interface. Search filters should reduce the effort needed to identify potential donors by blood group and location. Actual performance and usability should be assessed through testing with sample data and user feedback.

15. Limitations

- Donor contact details and availability may become outdated.
- The application cannot independently verify medical eligibility or guarantee a match.
- Location search depends on the accuracy of user-entered location information unless geospatial features are added.
- Real-world use requires clear consent, access controls, moderation, and an appropriate privacy policy.

16. Future Enhancements

- Map-based search with opt-in location sharing.
- Email or SMS notifications for verified requests, with donor consent.
- Donor availability toggle and periodic confirmation of profile details.
- Email/phone verification and administrator moderation.
- Pagination, sorting, and geospatial indexes for larger datasets.
- Accessible multilingual interface and mobile-friendly progressive web app.
- Analytics showing aggregate search activity without exposing individual donor data.

17. Conclusion

The MERN-Based Blood Donor Finder is a practical full-stack development project that combines a React interface, Node.js and Express REST APIs, and MongoDB data storage. Its main functions are donor registration, search by blood group and location, and controlled management of records. The project provides a useful setting to demonstrate CRUD operations, API design, frontend-backend integration, and authentication. Privacy, donor consent, and careful wording about medical suitability must remain central to the design.

References

- MongoDB Documentation — https://www.mongodb.com/docs/
- Express.js Documentation — https://expressjs.com/
- React Documentation — https://react.dev/
- Node.js Documentation — https://nodejs.org/docs/
- Mongoose Documentation — https://mongoosejs.com/docs/